

ASSESSMENT TOOL 3 – INPUT

NAME : R S JAHNAVI

REGISTER NO : 192371056

COURSE CODE : CSA1717

COURSE NAME : ARTIFICIAL INTELLIGENCE

Q1. A* SEARCH ALGORITHM (Dry Run with full trace)

```
import heapq
```

```
GRAPH = {  
    'A': {'B': 2, 'C': 4},  
    'B': {'A': 2, 'C': 3, 'D': 7, 'E': 2},  
    'C': {'A': 4, 'B': 3, 'E': 3},  
    'D': {'B': 7, 'E': 2, 'G': 2},  
    'E': {'B': 2, 'C': 3, 'D': 2, 'G': 2},  
    'G': {'D': 2, 'E': 2}  
}
```

```
HEURISTIC = {  
    'A': 7,  
    'B': 6,  
    'C': 4,  
    'D': 3,  
    'E': 2,  
    'G': 0
```

```
}
```

```
START = 'A'    # <-- change source node here
```

```
GOAL = 'G'     # <-- change destination node here
```

```
def astar_dry_run(graph, heuristic, start, goal):
```

```
    open_list = []
```

```
    heapq.heappush(open_list, (heuristic[start], 0, start, [start]))
```

```
    closed_list = set()
```

```
    g_scores = {start: 0}
```

```
    iteration = 1
```

```
    print("=" * 90)
```

```
    print(f"A* SEARCH DRY RUN : START = {start}, GOAL = {goal}")
```

```
    print("=" * 90)
```

```
    while open_list:
```

```
        open_snapshot = sorted(open_list, key=lambda x: x[0])
```

```
        open_display = [
```

```
            f"{n}(g={g},h={heuristic[n]},f={f})" for f, g, n, p in open_snapshot
```

```
        ]
```

```
        f, g, current, path = heapq.heappop(open_list)
```

```
        if current in closed_list:
```

continue

```
print(f"\nIteration {iteration}")
print(f"Current Node : {current}")
print(f"Open List    : {open_display}")
print(f"Closed List   : {sorted(closed_list)}")
print(f"g({current}) = {g}, h({current}) = {heuristic[current]}, "
      f"f({current}) = {f}")
```

iteration += 1

if current == goal:

```
    print("\n" + "=" * 90)
    print("GOAL REACHED!")
    print(f"Optimal Path    : {'->'.join(path)}")
    print(f"Total Path Cost : {g}")
    print("=" * 90)
    return path, g
```

closed_list.add(current)

for neighbor, cost in graph[current].items():

if neighbor in closed_list:

continue

tentative_g = g + cost

if neighbor not in g_scores or tentative_g < g_scores[neighbor]:

```

        g_scores[neighbor] = tentative_g
        f_neighbor = tentative_g + heuristic[neighbor]

        heapq.heappush(open_list, (f_neighbor, tentative_g, neighbor, path +
[neighbor]))

    print("No path found.")

    return None, None

astar_dry_run(GRAPH, HEURISTIC, START, GOAL)

```

Q2. Minimax with Alpha-Beta Pruning

```

TREE = [
    [3, 5, 6], # children of first MIN node
    [9, 1, 2] # children of second MIN node
]

pruned_nodes = []

def alphabeta(node, depth, alpha, beta, maximizing, path_label="ROOT"):
    indent = " " * depth

    if isinstance(node, int):
        print(f'{indent}Leaf {path_label} = {node}')
        return node

    if maximizing:
        value = float('-inf')

        print(f'{indent}Entering MAX node [{path_label}] (alpha={alpha}, beta={beta})')

```

```

for i, child in enumerate(node):
    child_label = f"{path_label}-{i + 1}"
    child_val = alphabeta(child, depth + 1, alpha, beta, False, child_label)
    value = max(value, child_val)
    alpha = max(alpha, value)
    print(f"{indent} -> After child {child_label}: value={value}, "
          f"alpha={alpha}, beta={beta}")
    if beta <= alpha:
        remaining = node[i + 1:]
        if remaining:
            print(f"{indent} ** PRUNING remaining children of {path_label}: "
                  f"{remaining} (beta <= alpha)")
            pruned_nodes.append((path_label, remaining))
            break
print(f"{indent} MAX node [{path_label}] selected value = {value}")
return value

```

```

else:
    value = float('inf')
    print(f"{indent} Entering MIN node [{path_label}] (alpha={alpha}, beta={beta})")
    for i, child in enumerate(node):
        child_label = f"{path_label}-{i + 1}"
        child_val = alphabeta(child, depth + 1, alpha, beta, True, child_label)
        value = min(value, child_val)
        beta = min(beta, value)
        print(f"{indent} -> After child {child_label}: value={value}, "
              f"alpha={alpha}, beta={beta}")
        if beta <= alpha:
            remaining = node[i + 1:]

```

```

    if remaining:
        print(f'{indent} ** PRUNING remaining children of {path_label}: '
              f'{remaining} (beta <= alpha)')
        pruned_nodes.append((path_label, remaining))
        break
    print(f'{indent} MIN node [{path_label}] selected value = {value}')
    return value

```

```

print("=" * 90)
print("MINIMAX WITH ALPHA-BETA PRUNING DRY RUN")
print("=" * 90)

```

```

best_value = alphabeta(TREE, 0, float('-inf'), float('inf'), True, "MAX")

```

```

print("\n" + "=" * 90)
print(f'FINAL MINIMAX VALUE : {best_value}')
print(f'PRUNED NODES      : {pruned_nodes if pruned_nodes else 'None'}')
print("=" * 90)

```